03

Arrays e laços for, for...of e forEach

Apostila do aluno · duração da aula: 2h15

O que você vai aprender

- 1. O que é um array?
- 2. Por que existe o for?
- 3. O for e suas três partes
- 4. for ou while?
- 5. Percorrendo um array com for
- 6. for...of: um item de cada vez
- 7. forEach: o array percorre a si mesmo
- 8. break e continue
- 9. for aninhado

Antes de começar

Esta aula usa o que vimos na **Aula 2**: `if`, `else` e principalmente o `while`. Todos os exemplos e exercícios podem ser executados direto no navegador, sem instalar nada, em **site-alunos-nu.vercel.app**.

Prof. Walter Caciano · Greenn School

1 O que é um array?

Até agora cada variável guardava **um** valor. Um **array** é uma variável que guarda uma **lista** de valores, em ordem.

Pense numa **fileira de gavetas numeradas**: cada gaveta guarda um valor, e o número da gaveta é o **índice**.

```
const frutas = ["maçã", "banana", "uva"];
//   índice:      0        1        2
```

O que você quer	Como escrever	Resultado
Pegar o primeiro item	`frutas[0]`	"maçã"
Saber quantos itens tem	`frutas.length`	3
Pegar o último item	`frutas[frutas.length - 1]`	"uva"
Trocar um item	`frutas[1] = "manga"`	["maçã", "manga", "uva"]
Adicionar no fim	`frutas.push("kiwi")`	agora com 4 itens

> **A contagem começa no zero.** O primeiro item é o `[0]` e o último é o `[length - 1]`. Um índice que não existe, como `frutas[10]`, devolve `undefined`.

2 Por que existe o `for` ?

Na Aula 2 vimos o **while**. Para repetir cinco vezes, precisamos lembrar de três coisas espalhadas pelo código:

```
let i = 1;          // 1. criar o contador
while (i <= 5) {    // 2. testar
  console.log(i);
  i++;              // 3. atualizar (esquecer disso = loop infinito)
}
```

O **for** junta essas três coisas na **mesma linha**. Por isso ele é o laço mais usado quando sabemos quantas voltas queremos dar.

3 O `for` e suas três partes

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Parte	No exemplo	O que faz
Inicialização	`let i = 1`	roda **uma vez**, antes de tudo
Condição	`i <= 5`	testada **antes de cada volta**; se for false, o laço acaba
Atualização	`i++`	roda **no fim de cada volta**

Leia em voz alta: "*começando em 1, enquanto i for menor ou igual a 5, aumente 1 a cada volta.*"

> **Variações úteis:** `i += 2` anda de 2 em 2; `i--` faz contagem regressiva.

4 `for` ou `while` ?

	for	while
Melhor quando	você **sabe** quantas voltas: de 1 a 10, os itens de uma lista	a parada depende de algo que **muda**: enquanto tiver saldo
Contador	tudo dentro dos parênteses	espalhado pelo código

Os dois repetem do mesmo jeito. É questão de qual fica mais claro de ler.

5 Percorrendo um array com `for`

Como o índice começa em 0 e vai até `length - 1`, o for de array usa `<`, e não `<=`:

```
for (let i = 0; i < frutas.length; i++) {
  console.log(`${i + 1}. ${frutas[i]}`);
}
```

6 `for...of` : um item de cada vez

Leia assim: "*para cada **fruta** de **frutas**, faça...*". A cada volta, a variável `fruta` recebe o **próximo item** da lista. Não tem contador nem índice, então não tem como errar o `<`.

```
for (const fruta of frutas) {
  console.log(fruta);
}
// 1ª volta: fruta = "maçã"
// 2ª volta: fruta = "banana"
// 3ª volta: fruta = "uva"  -> acabou a lista, acabou o laço

for (const letra of "JS") {
  console.log(letra);   // também funciona com texto: J, depois S
}
```

7 `forEach` : o array percorre a si mesmo

O **forEach** é um método do array: em vez de escrever um laço, você pede para a própria lista executar um bloco **para cada item**.

```
frutas.forEach((fruta) => {
  console.log(fruta);
});

frutas.forEach((fruta, i) => {      // o 2º valor é a posição
  console.log(`${i + 1}. ${fruta}`);
});
```

Leia assim: "*frutas, para cada item (que vou chamar de fruta), faça isto.*" A parte `(fruta) => { }` é uma **função**, que vamos estudar mais pra frente. Por enquanto, use como receita: o nome do item entre parênteses, depois `=>`, depois o bloco.

	for com índice	for...of	forEach
Tem a posição?	sim, `i`	não	sim, 2º valor
Aceita break/continue?	sim	sim	**não**
Funciona com texto?	sim	sim	não, só array
Quando usar	precisa do índice ou de controle total	só precisa dos valores	percorrer a lista inteira de forma curta

> **Atenção:** `break` e `continue` não funcionam no forEach: ele sempre percorre a lista inteira. Para parar no meio, use for ou for...of.

8 `break` e `continue`

break sai do laço na hora. **continue** pula o resto da volta e vai para a próxima.

```
for (const n of [4, 9, 7, 2]) {
  if (n === 7) {
    console.log("Achei o 7!");
    break;                      // não olha o resto
  }
}

for (let i = 1; i <= 5; i++) {
  if (i % 2 !== 0) continue;    // pula os ímpares
  console.log(i);               // 2, 4
}
```

9 `for` aninhado

Um for dentro do outro: o de dentro roda **inteiro** a cada volta do de fora.

```
for (let linha = 1; linha <= 3; linha++) {
  let texto = "";
  for (let coluna = 1; coluna <= linha; coluna++) {
    texto += "*";
  }
  console.log(texto);   //  *   **   ***
}
```

> Use nomes que expliquem o papel de cada contador (`linha` , `coluna`) em vez de `i` e `j` .

10 Exercícios

Resolva na ordem: os primeiros treinam o `for` e os últimos juntam tudo. Em cada um, compare a sua saída com a esperada — cada letra, espaço e acento conta.

01 Contagem — NÍVEL 1

Use um for para mostrar os números de 1 até 5, um por linha.

Saída esperada: 1 2 3 4 5, um por linha

02 Tabuada do 7 — NÍVEL 1

Use um for de 1 a 10 e mostre a tabuada do 7.

Saída esperada: 7 x 1 = 7 ... 7 x 10 = 70

03 Soma de 1 a 100 — NÍVEL 1

Crie `let soma = 0`, some cada número de 1 a 100 dentro do for e mostre o total depois do laço.

Saída esperada: A soma é 5050

04 Só os pares — NÍVEL 1

Mostre os pares de 0 até 20. Faça de duas formas: com `i += 2` e com um `if` dentro do for.

Saída esperada: 0, 2, 4 ... 20

05 Contagem regressiva — NÍVEL 1

Conte de 10 até 1 com for e, depois do laço, mostre "Fogo!".

Saída esperada: 10, 9 ... 1, Fogo!

06 Média das notas — NÍVEL 2

Com `const notas = [8, 6.5, 9, 7]`, some tudo com um laço e mostre a média com uma casa decimal (`.toFixed(1)`).

Saída esperada: Média: 7.6

07 Lista numerada com forEach — NÍVEL 2

Com `const frutas = ["maçã", "banana", "uva"]`, use `frutas.forEach((fruta, i) => { ... })` para mostrar a lista numerada. Lembre que a posição começa em 0.

Saída esperada: 1. maçã / 2. banana / 3. uva

08 Contando letras

NÍVEL 2

Use for...of para contar quantas vezes a letra "a" aparece em "programação".

Saída esperada: A letra a aparece 2 vezes

09 Procurando com break

NÍVEL 3

Com `const numeros = [4, 9, 7, 2, 7]`, pare no primeiro 7 e mostre a posição dele.

Saída esperada: Achei o 7 na posição 2

10 Pulando com continue

NÍVEL 3

Com `const valores = [5, -2, 8, -1, 3]`, some só os positivos usando continue.

Saída esperada: Soma dos positivos: 16

11 O maior da lista

NÍVEL 3

Com `const numeros = [12, 45, 7, 89, 23]`, descubra o maior valor usando um laço (sem Math.max).

Saída esperada: O maior é 89

12 Pirâmide de estrelas

DESAFIO

Usando dois for, desenhe uma pirâmide de 5 linhas de estrelas.

Saída esperada: * / ** / *** / **** / *****

Travou? Volte ao exemplo da seção correspondente, rode o código no site e mude um valor por vez para entender o que cada parte faz.